

RTOS Experiment 10

Counting Semaphore using FreeRTOS

Platform

STM32 Nucleo-F446RE

Framework

FreeRTOS Synchronization

Focus

Resource Sharing & Task
Scheduling

Aim of Experiment

This experiment explores the practical implementation of FreeRTOS counting semaphores on the STM32 Nucleo-F446RE platform, focusing on how multiple tasks share resources in a real-time environment.

1

Implement Semaphore

Build a FreeRTOS counting semaphore from scratch on embedded hardware.

2

Study Shared Access

Observe how multiple tasks access a single shared resource concurrently.

3

Task Synchronization

Analyse how the RTOS scheduler coordinates task execution timing.

4

Interleaving Analysis

Examine the interleaved output patterns produced by concurrent tasks.

Introduction to RTOS

What is RTOS?

A **Real-Time Operating System** executes multiple tasks simultaneously while providing deterministic, time-bound behaviour. It relies on scheduling and synchronization techniques to manage concurrent workloads reliably.

Need for Synchronization

When multiple tasks share resources, synchronization becomes critical. Without it, systems face:

- Resource conflicts between tasks
- Data integrity violations
- Race conditions causing unpredictable behaviour

by David Kalinsky (Nov. 18, 2003)

The fundamentals

To most people, embedded systems are not recognizable as computers. Instead, they are hidden inside everyday objects that surround us and help us in our lives. Embedded systems typically do not interface with the outside world through familiar personal computer interface devices such as a mouse, keyboard and graphic user interface. Instead, they interface with the outside world through unusual interfaces such as sensors, actuators and specialized communication links.

Real-time and embedded systems operate in constrained environments in which computer memory and processing power are limited. They often need to provide their services within strict time deadlines to their users and to the surrounding world. It is these memory, speed and timing constraints that dictate the use of real-time operating systems in embedded software.

Basic kernel services

In the discussion below, we will focus on the "kernel" – the part of an operating system that provides the most basic services to application software running on a processor.

The "kernel" of a real-time operating system ("RTOS") provides an "abstraction layer" that hides from application software the hardware details of the processor (or set of processors) upon which the application software will run. This is shown in Figure 1.

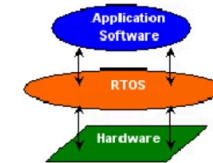


Figure 1: An RTOS Kernel provides an Abstraction Layer between Application Software and Embedded Hardware

In providing this "abstraction layer" the RTOS kernel supplies five main categories of basic services to application software, as seen in Figure 2.

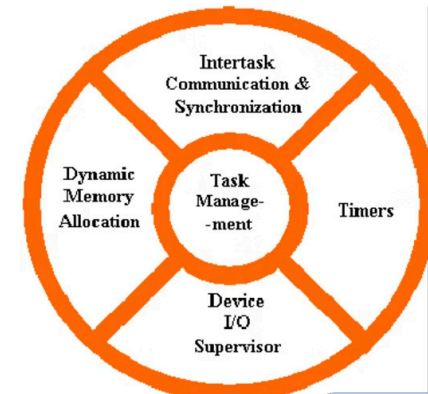


Figure 2: Basic Services Provided by a Real-Time

Counting Semaphore

A counting semaphore is a core synchronization mechanism in RTOS that allows **limited concurrent access** to a shared resource by maintaining an internal counter value.



Acquire / Take

A task decrements the counter to claim access. If the counter is zero, the task is blocked until a slot is freed.



Release / Give

A task increments the counter upon completion, signalling that a resource slot is now available for other tasks.



Resource Management

Controls how many tasks may access a resource simultaneously, improving throughput and enabling controlled multitasking.

Hardware and Software Used

Hardware Apparatus

STM32 Nucleo-F446RE

Primary microcontroller development board

USB Cable

Programming and power interface

Ultrasonic Sensor

Peripheral device for hardware interaction

Software Tools

STM32CubeMX

Graphical configuration and code generation tool

STM32CubeIDE

Integrated development and debugging environment

FreeRTOS

Open-source real-time operating system kernel

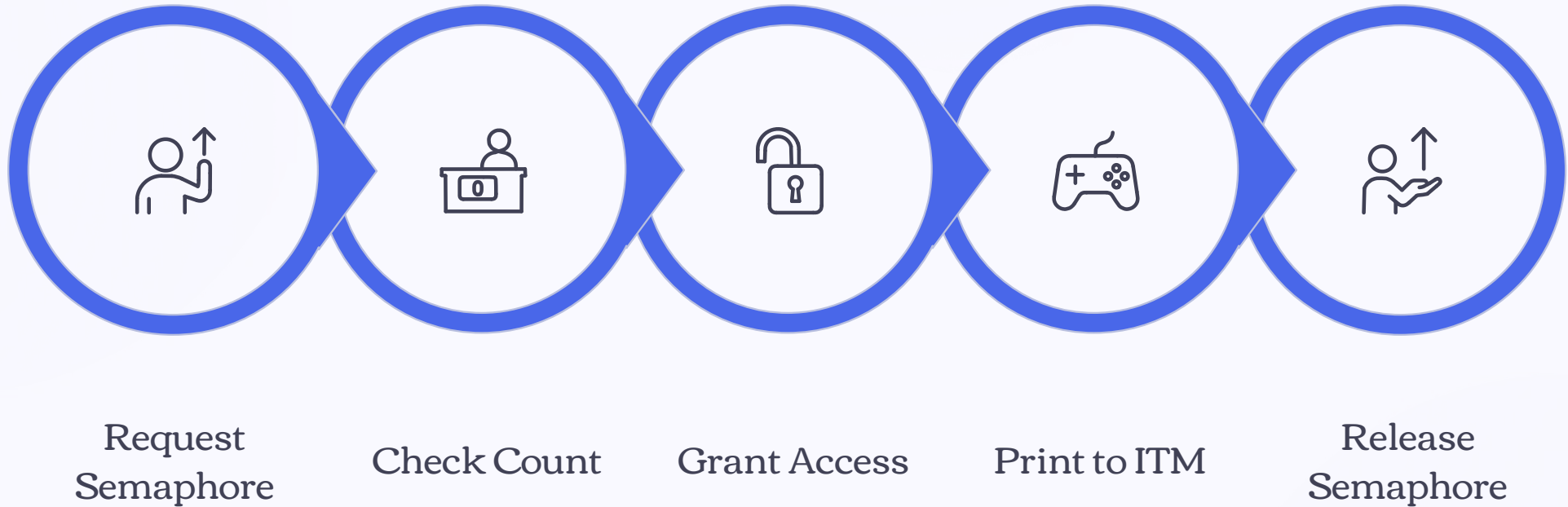
Task Architecture

The experiment creates three concurrent tasks – **Task A**, **Task B**, and **Task C** – all competing for access to the shared **ITM Trace Console**, controlled via a counting semaphore.



- ⓘ The counting semaphore acts as the sole access controller for the shared ITM Trace Console resource across all three tasks.

Program Flow



The RTOS kernel manages this flow using **context switching** and **preemptive scheduling**, allowing tasks to be paused and resumed based on semaphore availability and priority, ensuring efficient CPU utilisation across all active tasks.

Output Observation

Due to multiple tasks writing simultaneously to the shared ITM Trace Console, the observed output shows **mixed and interleaved character patterns** rather than clean sequential output.

Pattern 1

ACACAC...

Tasks A and C interleaving on the console

Pattern 2

BABABA...

Tasks B and A alternating in output

Pattern 3

CBCBCB...

Tasks C and B sharing the channel

Result and Analysis

Result

The counting semaphore was **successfully implemented** on the STM32 Nucleo-F446RE. Resource sharing across three concurrent tasks was achieved, with all tasks gaining access to the ITM Trace Console in an interleaved manner.

Key Analysis Findings

- Increased throughput observed with concurrent access
- Task interleaving clearly visible in console output
- No strict mutual exclusion – multiple tasks accessed simultaneously

⚠ A counting semaphore **controls quantity** of access but does **not guarantee atomic access**. Use a Mutex for exclusive resource ownership.

Conclusion

This experiment successfully demonstrated the use of **counting semaphores** in FreeRTOS for managing shared resource access across multiple concurrent tasks on the STM32 Nucleo-F446RE platform.



Efficient Resource Management

Counting semaphores effectively manage how many tasks access a shared resource simultaneously, improving overall system throughput.



Scheduling Impacts Execution Order

RTOS preemptive scheduling directly influences the interleaving patterns observed in task output, as seen in the console results.



Mutexes for Exclusive Access

For scenarios requiring strict mutual exclusion, Mutexes are the preferred synchronization primitive over counting semaphores.



Experiment objective achieved – synchronization and task interleaving successfully demonstrated using FreeRTOS counting semaphores.